September 24, 2026

DSK809 Deep Learning

Week 41 - CNNs

Exam: We finally settled on the exam format. The exam will feature a conversation of the course curriculum with an outset in your submitted exercise portfolio, and will expand into the rest of the curriculum for further discussion. It should be pretty approachable, but if you prefer a more traditional oral exam format, please express your preference in the midterm evaluation (available soon).

Lecture

The next deep learning lecture is scheduled for October 1st at 12:30. This session will cover convolutional neural networks and other foundational aspects of computer vision.

Chapter 9 in the Deep Learning book has most of the essential material. Have a look through for any of the specifics.

- Goodfellow et al. 2016, *Deep Learning*, Chapter 9. link

Keywords: Convolutional neural network, convolution and pooling, kernels and filters, transfer learning, residual layers.

Anton D. Lautrup ♦ *lautrup@imada.sdu.dk*

Exercises

Following the lecture, we will have the exercise session, where I will be available for general questions and one-on-one sparring. You will work in groups or on your own, on the problems described below.

Getting started

- What problem do CNNs solve better than fully connected networks?
- How do convolutional layers reduce the number of parameters compared to dense layers?
- How do pooling layers contribute to translation invariance?
- What is the difference between max pooling and average pooling?
- How can image augmentation be used as regularisation?

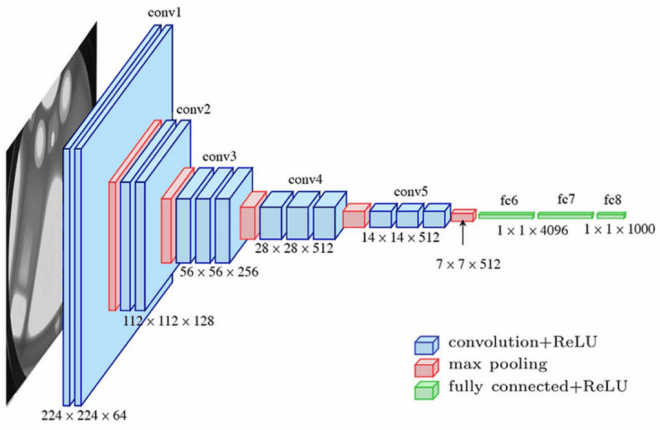


Figure 1: VGG-16 model

Practical problems

1. **Loading and using pre-trained models**
   Working in deep learning is often about knowing when to choose one's battles, competing with big tech, on making the actually big models, is very difficult; and often, using existing big models can get you a long way.

   (a) Look at Figure 1, and make a computation of roughly how many trainable parameters are in the model.

   (b) Use the code provided in `convolutions.ipynb`, load the VGG16 model with pre-trained weights. Get the number of trainable parameters. How well did it match your estimate?

   (c) VGG16 is trained for ImageNet, which is a huge dataset. On GitHub, you can find some examples. Download and import a couple into your code space, and apply your instance of the VGG16 model to get predictions.

   (d) What are the possible predictions from the VGG16 model? What happens if you give it a picture from outside of this scope?

   (e) I put some code for showing how the image is processed in parallel channels, with learned convolution filters. Explain the patterns that you can observe.

2. **Fine-tuning a large model** **[Eligible for project]**
   In the previous exercise, we saw how a multi-million-parameter model could be applied. However, let's now consider the case where we want to apply the model to a completely different dataset. While the VGG16 model is impressive, we will use a smaller model, namely, ResNet18, which

is also trained on ImageNet. We will adjust the model to work on the CIFAR-10 dataset (163MB) and evaluate the performance over multiple rounds of fine-tuning. **N.B.** This task can be hardware-intensive.

(a) Import the pretrained ResNet18 model. Import the CIFAR-10 dataset, and note down the current performance of the model.

(b) You should be able to figure out how to overwrite the last "fc" layer with a linear layer of dims 512x10, or another sequential block of your own design. Test your model again. What performance do you now expect at this stage?

(c) Complete the missing elements in the training loop. Notice that we freeze the existing weights of the ResNet model and unfreeze only the last (untrained) block that we modified. What is the performance of the model after fine-tuning for 1 epoch, for 5 epochs, and for 10 epochs?

(d) Now, rather than replacing the fully connected output layer, we could instead want to build on top of it. Go back to the top of your code, and adjust the part where you replaced the "fc" layer. Remember to comment out the previous code. Explain how this affects your results. Is there any reasoning for why it might?

3. **Convolution and pooling** **[Eligible for project]**
In this exercise, you must implement functions for conducting convolution and pooling in Python. Your solutions are compared to those functions in PyTorch. Use the code provided in `convolutions.ipynb`, and the `lena512_gray.bmp` image as a base. Start by running both of the code frames to see what the output from PyTorch looks like.

(a) Explain briefly the convolution and pooling operations. What is the effect of the kernel (filter) and padding?

(b) Complete the code inside the `conv2d` function and see how your results compare with the torch implementation.

(c) Next, do the same for the `max_pool2d` function.

(d) Try using a different kernel for your convolution. [Hint: see Wikipedia on image processing kernels.]

Theoretical problems

4. **Information and dataset augmentation** **[Eligible for project]**
Information theory is a branch of computer science concerned with the mathematical study of the quantification, storage, and communication of information. In deep learning, particularly for image models, it is common to amplify the training data with rotated, cropped, and blurred examples to foster increased sample value. Arguably, we cannot do this indefinitely; at some point, the information gain will be insignificant. This exercise

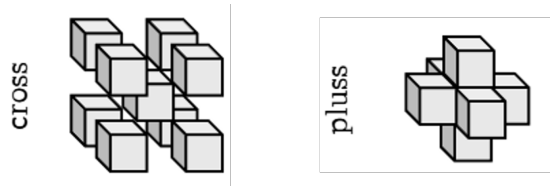


Figure 2: Crosses and plusses

explores the point at which we start to see diminishing returns.
One way to measure the unique information content of a dataset is by the difference between the total information and the redundant information. By using entropy and mutual information as proxies for these quantities, we can measure:

$$\mathcal{U} = \sum_i \mathcal{H}(d_i) - \sum_{i \neq j} \mathcal{I}(d_i, d_j) \tag{1}$$

(a) In the codebook, complete the methods for computing image entropy and image mutual information. [Hint: I do not expect that you know how to implement these functions outright. This is a "search around for a solution that works, and then figure out how and why it works post hoc"-exercise (extra tips in the codebook).]

(b) Add 7 different transformations to the list of transformations.

(c) The final function should plot a graph of the total information, mutual information, and unique information (accumulated). Discuss what you see. What is the variation, and are some kinds of transformations more effective than others?

(d) What we measured was if the new information was unique, but is "uniqueness" the only criterion we are interested in for data augmentation? [Hint: Is noise unique?]

Challenge problem

5. **3D convolutional neural network** **[Eligible for project]**
This exercise is very free-form. Download the `crosses_and_pluses.h5` file from Itslearning (see Figure 2). It is your task to build an effective convolutional model for this dataset. We provided a little code for you to get started with the dataset. For the delivery, in addition to relevant codebits, provide an illustration of the model you designed, see for example Figure 1.